Relatório de evolução -- Sprint 4

Chatbot ChargeGrid Intelligence -- RAG medido, confiável e utilizável EV Challenge -- GoodWe / FIAP · Prompt and Artificial Intelligence · 2026.2 · Turma 1CCPG

1. Resumo da evolução

Nas Sprints 1 e 2 o chatbot respondia com base em 22 frases soltas da planilha SP2, encontradas por palavra-chave, sem dizer de onde vinha cada informação. Na Sprint 3 o núcleo foi reescrito em LangChain, com memória por tokens, saída estruturada e cinco camadas de segurança, mas a busca continuou a mesma.

Na Sprint 4 a busca virou um RAG de verdade: dez documentos do domínio (manual do carregador GoodWe HCA, material do Secovi-SP sobre condomínios, guias da Arval e do PROMOB-e, e os documentos do próprio ChargeGrid) são divididos em chunks, vetorizados com nomic-embed-text e guardados num ChromaDB persistente. Toda resposta cita documento e seção, pergunta sem resposta na base é recusada em código, e instrução escondida dentro de documento é removida antes de chegar ao modelo. O resultado é medido em três iterações -- faithfulness e answer relevancy pela rubrica manual de fallback (evals/fallback/rubrica_manual.md), aplicada por um LLM-juiz -- e aparece numa interface web (Gradio) com as fontes visíveis.

2. Pipeline RAG

```
PyMuPDFLoader / Markdown -> RecursiveCharacterTextSplitter -> nomic-embed-text -> ChromaDB
pergunta -> (reescrita se encadeada) -> busca com filtro de acesso e limiar -> blindagem
         -> prompt RAG versionado -> modelo -> guarda de saída -> citação garantida
```

Base de conhecimento. 5 PDFs públicos (GoodWe, Secovi-SP x2, Arval, PROMOB-e) e 5 documentos escritos para o projeto (manual de operação e tabela tarifária do ChargeGrid, FAQ de recarga, modelo de regimento de condomínio e o relatório histórico SP2, gerado da planilha real). Cada documento tem metadados de tipo, nível de acesso e link de origem; o relatório SP2 é acesso: gestao e só entra na busca para o perfil de gestão.

Chunking. Comparamos duas estratégias:

- fixo_1000 -- 1000 caracteres, overlap 150, sobre o texto cru da página.
- secao_500 -- 500 caracteres, overlap 75, dentro de cada página ou seção, com o título do documento e a seção no início de cada chunk.

A iteração 1 usa fixo_1000 e as iterações 2 e 3, secao_500; a escolha final segue a recuperação (hit@k) e o RAGAS da tabela da seção 3. O argumento a favor do chunk menor: uma página do manual GoodWe mistura instalação, LEDs e app, e um chunk de 1000 caracteres carrega tudo isso junto, diluindo a similaridade; o cabeçalho "documento -- seção" ajuda a busca e dá a citação pronta. O custo é ter mais chunks (493 contra 295). O overlap de 15% faz a frase cortada no limite aparecer inteira num dos vizinhos.

Parâmetros. k = 4 trechos, limiar de relevância 0,45 (abaixo disso o trecho é descartado; calibrado por evals/calibrar_limiar.py), temperatura 0. Detalhes e comparação de modelos em docs/relatorio_modelos.md.

3. Comparativo antes/depois

Como medimos. Mesmo eval set (evals/eval_set_rag.json, 24 casos: 16 com resposta nos documentos, 2 de gestão, 2 perguntas encadeadas, 2 sem resposta na base, 1 fora de escopo e 1 de dado restrito), mesmo modelo respondendo, mesmo juiz e mesma rubrica para todas as versões.

- Avaliação: rubrica manual aplicada por LLM-juiz, no lugar do RAGAS. O RAGAS precisa do juiz e dos embeddings ao mesmo tempo, com chamadas em paralelo, e não terminava dentro do limite da conta gratuita. Usamos então a rubrica de fallback (evals/fallback/rubrica_manual.md), com notas de 0 a 1 em passos de 0,25 para faithfulness e answer relevancy. A equivalência com o RAGAS está justificada no próprio arquivo. O juiz recebe pergunta, trechos e resposta e devolve as duas notas. Recusar uma pergunta que tinha resposta na base vale faithfulness 1 e relevancy 0, regra aplicada em código, igual para todas as versões. O juiz (temperatura 0, max_tokens 1500) aparece logo abaixo da tabela de parâmetros.
- Embeddings no Ollama local. O nomic-embed-text rodou num servidor Ollama instalado dentro do Colab (http://127.0.0.1:11434), porque a Ollama Cloud recusava as chamadas de embedding (HTTP 401). É o mesmo modelo, com os prefixos search_document:/search_query:. Só muda onde ele roda.

- Modelos e parâmetros de cada etapa (lidos dos arquivos de resultado):

Etapa	temperature	top_p	max_tokens	k	Prompt
Sprints 1/2	0,4	1,0	450	5	legado (Sprints 1/2)
Iteração 1	0,0	1,0	450	3	rag_v1
Iteração 2	0,0	1,0	450	4	rag_v2
Iteração 3	0,0	1,0	450	4	rag_v3

Modelo que respondeu em todas as etapas: groq:openai/gpt-oss-20b. Juiz da rubrica: gemini:gemini-3.1-flash-lite.

Embeddings registrados nas execuções: nomic-embed-text via Ollama em http://127.0.0.1:11434.

Tabela antes/depois (Sprints 1/2 x Sprint 04, a coluna da Sprint 04 é a iteração 3):

Critério	Sprints 1/2 (versão original)	Sprint 04 (RAG avaliado)
Recuperação	palavra-chave em 22 frases da planilha SP2	busca vetorial (nomic-embed-text + ChromaDB) em 10 documentos, com filtro de acesso e limiar
Faithfulness	0,350	0,988
Faithfulness por iteração	versão única	it1 0,84 -> it2 0,99 -> it3 0,99
Answer relevancy	0,812	0,750
Answer relevancy por iteração	versão única	it1 0,68 -> it2 0,72 -> it3 0,75
Qualidade do contexto recuperado (documento certo entre os trechos)	10% das perguntas trouxeram algum contexto (sem documento a conferir)	85%
Presença de citação de fonte	0%	100%
Recusa correta (sem resposta na base, fora de escopo, dado restrito)	100%	75%
Checagens determinísticas OK	46%	67%
Latência média por turno	0,59 s	9,98 s
Tokens por turno (média)	1289	2475

Faithfulness e answer relevancy medidos com: rubrica de evals/fallback/rubrica_manual.md aplicada por LLM-juiz gemini:gemini-3.1-flash-lite. Modelo que respondeu: groq:openai/gpt-oss-20b.

Iterações do RAG (o ganho de cada uma é atribuído à mudança listada):

- Iteração 1: chunk fixo 1000/150, prompt rag_v1, k=3, sem guardas em código.
- Iteração 2: chunk por seção 500/75 com cabeçalho, prompt rag_v2 (grounding + citação), limiar 0,45, recusa em código.
- Iteração 3: prompt rag_v3 com exemplos + reescrita da pergunta encadeada.

Iteração	Faithfulness	Ganho	Answer relevancy	Ganho	Recuperação (hit@k)	Citação	Modelo citou [n]	Recusa correta
Iteração 1	0,838	--	0,675	--	75%	0%	0%	100%
Iteração 2	0,988	+0,150	0,725	+0,050	85%	100%	100%	100%
Iteração 3	0,988	+0,000	0,750	+0,025	85%	100%	93%	75%

Faithfulness e answer relevancy medidos com: rubrica de evals/fallback/rubrica_manual.md aplicada por LLM-juiz gemini:gemini-3.1-flash-lite. Modelo que respondeu: groq:openai/gpt-oss-20b.

Leitura dos resultados. Do chatbot antigo para a iteração 1, a faithfulness sobe de 0,35 para 0,84. O legado respondia "de cabeça": em 13 dos 20 casos com resposta na base nenhum trecho sustentava o que ele disse. A answer relevancy cai de 0,81 para 0,68, e isso tem uma causa: o legado responde tudo (bem ou mal), enquanto a iteração 1 recusa quando a busca não traz o documento certo. Os 5 casos sem recuperação da iteração 1 (hit@k 75%) são todos do manual GoodWe HCA (grau IP, modos de recarga, corrente mínima, carro monofásico e a encadeada "e o do plugue?"). Em 4 deles o modelo recusou, e recusa a uma pergunta que tinha resposta vale relevancy 0 pela rubrica. No quinto respondeu sem apoio nos trechos e levou 0 nas duas notas. A iteração 2 ataca exatamente essa falha de recuperação: com o chunk por seção e o cabeçalho do documento, o hit@k vai de 75% para 85%, a faithfulness de 0,84 para 0,99 e a relevancy de 0,68 para 0,73. A citação passa a sair em 100% das respostas, e o próprio modelo cita [n] em todas. A iteração 3 (exemplos no prompt + reescrita da pergunta encadeada) mantém a faithfulness e sobe a relevancy para 0,75, o segundo ganho seguido. Ela também tem um custo: a recusa correta cai de 100% para 75% (uma das duas perguntas sem resposta na base recebeu resposta, montada com um trecho que não a respondia) e o modelo passa a citar [n] em 93% das respostas (a citação final continua em 100%, porque é garantida em código). A relevancy final (0,75) ainda fica abaixo da do legado (0,81) pelo motivo já explicado: o legado responde tudo, e cada recusa a pergunta com resposta na base vale 0. Em troca, o que o assistente diz agora é sustentado pelos documentos (0,99 contra 0,35) e sempre vem com a fonte. Latência e tokens por turno sobem (0,59 s -> 9,98 s; 1289 -> 2475) por causa da busca, dos trechos no prompt e da reescrita. A comparação dos quatro modelos na configuração da iteração 3 está em docs/relatorio_modelos.md: todos ficam entre 0,99 e 1,00 de faithfulness, e só o gpt-oss-20b falha uma recusa.

Segurança:

Teste	Resultado
Injeção via documento -- com blindagem	resistiu em 3/3
Injeção via documento -- sem blindagem (só o prompt)	resistiu em 2/3
Bateria da Sprint 3 (24 casos, 12 ataques)	checagens 71%, ataques recusados 100%

Os números saem de evals/resultados/*.json e são reproduzidos com python -m evals.rodar_tudo (rubrica de fallback) ou, com RAGAS, python -m evals.run_legado_rag, python -m evals.run_iteracoes e python -m

evals.run_modelos.

4. Problemas encontrados e soluções

PDF diagramado saía uma palavra por linha. O guia da Arval é um infográfico: o PyMuPDF devolvia cada palavra numa linha separada ("Os", "veículos", "elétricos"...), e o splitter cortava nessas quebras, gerando chunks sem sentido. Passamos a juntar quebras simples de linha nos PDFs (linha em branco continua separando parágrafo). A apresentação do Secovi de 2026 é quase toda imagem (4,7 mil caracteres em 40 páginas); mantivemos na base, mas ela pouco contribui -- OCR ficou como próximo passo.

A blindagem de documento barrava texto normal. A primeira versão reaproveitava os padrões de ataque da Sprint 3 e marcou "o Secovi acompanhará as novas diretrizes" como instrução injetada. Documento técnico fala de regra, limite e acesso o tempo todo, então escrevemos padrões próprios, que só pegam frase dirigida ao assistente ("nova instrução:", "o assistente deve...", "[SYSTEM]"). O teste test_base_real_sem_falso_positivo roda a blindagem nos 493 chunks reais e exige zero remoções.

Pergunta encadeada não achava nada. Limitação anotada na Sprint 3: "e no horário de ponta?" não tem termo para buscar. Com a recusa em código da Sprint 4 isso piorava, porque a busca vazia virava "não encontrei". Agora, quando há histórico e a pergunta é curta ou começa com "e", "esse", "dele"..., o modelo reescreve a pergunta completa antes da busca (iteração 3).

Relatório comercial podia vazar para o motorista. Na Sprint 3 a fronteira de acesso era um if na montagem do contexto. Com tudo num banco vetorial só, um trecho do relatório SP2 poderia aparecer para qualquer pergunta parecida. A solução foi o metadado acesso em cada documento e o filtro where na busca do Chroma: sem perfil de gestão, o relatório nem entra no ranking.

5. Integrantes -- Turma 1CCPG

Nome	RM
Luan de Araujo Carneiro	573691
Pedro Sampaio Mochnacs Arruda	573522
Raul Sampaio Mochnacs Arruda	573523
Pedro Ribeiro Lopes	570083
Kevin Rodrigues de Melo	571777
Pedro Vianna	570747
Lana Ozeki	569795